

Design and Analysis of Algorithms

Section 4: Fundamentals of Algorithm Analysis

4.1) Basic Concepts of Algorithm Analysis

- When we have a problem to solve, it is interesting to find several algorithms for the problem, then we can choose the best

في تحليل الخوارزميات (Algorithm Analysis)، عادةً ما يكون الغرض أو الهدف هو تحديد أفضل خوارزمية بين مجموعة من الخوارزميات التي تعمل على حل نفس المسألة.

فعلى سبيل المثال، فإن خوارزمية البحث المتسلسل (Linear Search) تتطلب زمن يتوقف على عدد الأعداد المدخلة. وبالنسبة للذاكرة فإن الخوارزمية تتطلب عدد من المرات للوصول الى الذاكرة يتوافق مع عدد الأعداد المدخلة.

لنفس مسألة الخوارزمية أعلاه قد تكمل خوارزميات أخرى (مثل خوارزمية البحث الثنائي) المهمة في وقت أو مساحة ذاكرة أو جهد أقل

- The choice between algorithms comes down to questions of efficiency
 - Which one takes the least of computing time?
 - Which one does the job with the least amount of memory space?

4.1) Basic Concepts of Algorithm Analysis

- **Definitions:**

- **Algorithm efficiency** is a term that refers to the amount of time or memory space required to execute algorithms.
- **Algorithm complexity** is a synonym term for algorithm efficiency
- **Algorithm analysis** is a term that refers to the process of deriving estimates for the time and space that needed to execute algorithms

- So, how can we do this complex task of algorithm analysis?

عليه سيدور في ذهننا سؤال، وهو كيف نتمكن من انجاز هذه العملية المعقدة للتحليل الخوارزميات؟

- There are three techniques for the estimation of algorithm efficiency: **Empirical approach**, **theoretical approach**, and **hybrid approach**.

4.2) Techniques of Algorithm Analysis

- The **empirical approach** consists of programming the competing algorithms and trying them on different instances with the help of a computer. The one with the least execution time will clearly be the best algorithm.
- The **theoretical approach** consists of determining mathematically the quantity of resources (execution time, memory space, and etc.) needed by each algorithm as a function of the size of the instances considered.
- It is also possible to analyze algorithms using a **hybrid approach**, where the form of the function describing the algorithm's efficiency is determined theoretically, and then any required numerical parameters are determined empirically for a particular program and machine, usually by some form of regression.

4.2) Techniques of Algorithm Analysis

- **Theoretical Technique of Algorithm Analysis:**

- ✓ The theoretical approach consists of *determining mathematically* the quantity of resources (execution time, memory space, and etc.) needed by each algorithm as a **function of the size** of the instances considered.
- ✓ The term **size** of the instances corresponds formally the **number of bits** needed to represent the instance on a computer.
- ✓ To make the analysis feasible, this term is used to refer an **integer** that in some way **measures the number of components in an instance**.

For example: Sorting an instance involving n items is generally considered to be of size n

- ✓ For **numerical algorithms**, the value of an instance is considered to the size of the algorithms.

For example: Sorting an instance involving n items is generally considered to be of size n

4.3) Asymptotic Notation (الترميز المقارب)

- Asymptotic Notation is a **mathematical tool** that allows us to represent an algorithm's execution time
- It **doesn't determine** the **exact number of** the **execution times** for algorithms, but it estimates the growth of the execution times due to the increase in the size of the algorithms.

الترميز المقارب هو عبارة عن **أداة رياضية** (Mathematical tool) تستخدم لوصف وقت تنفيذ الخوارزمية، حيث أنه يعبر عن الزيادة في وقت التنفيذ الناتجة عن الزيادة في حجم المدخلات (Size).

رمزياً يشير الترميز المقارب الى عملية رياضية تسمى بعملية **التحليل المقارب** (Asymptotic analysis).

4.3) Asymptotic Notation (الترميز المقارب)

- Asymptotic Notation doesn't determine the exact number of the execution times for algorithms, but it estimates the growth of the execution times due to the increase in the size of the algorithms.

يستخدم التحليل المقارب (Asymptotic analysis) من أجل حساب وقت التشغيل لأي عملية.

- فعلى سبيل المثال، يتم حساب وقت تشغيل إحدى العمليات على أنها $f(n) = n$
- وقد يحسب كذلك وقت التشغيل لعملية أخرى على أنه $g(n) = n^2$
- هذا يعني أن وقت التشغيل الأول سيزداد خطياً مع الزيادة في n ، بينما زيادة وقت التشغيل للعملية الثانية يزداد بإطراد (بشكل أسي exponentially)
- وبالمثل، سيكون وقت تشغيل كلتا العمليتين متماثلاً تقريباً إذا كانت n صغيرة جداً

4.3) Asymptotic Notation (الترميز المقارب)

- Asymptotic Notation doesn't determine the exact number of the execution times for algorithms, but it estimates the growth of the execution times due to the increase in the size of the algorithms.
- Therefore, it provides us with a measure for different computers, compilers, operating systems.

لا تبدو هنالك مشكلة عند استخدام أنواع مختلفة من الحواسيب أو أنظمة التشغيل (Operating Systems) أو المترجمات (Compilers). فعند كل تلك الاستخدامات المختلفة يقدم لنا الترميز المقارب (Asymptotic notation) المقياس الذي نستخدمه في تقدير زمن تنفيذ الخوارزميات

- We use **three types of asymptotic notations**:
 - 1) Big Oh (O -notation)
 - 2) Big Omega (Ω -notation)
 - 3) Big Theta (Θ -notation)

4.3) Asymptotic Notation (الترميز المقارب)

Big-O Notation:

- Denoted as follows

$$O(g)$$

Such that g is a function

- Read as follows

In the order at most of g

4.3) Asymptotic Notation (الترميز المقارب)

Big-O Notation:

It is formally defined as follows:

$$f(x) = O(g(x)) \leftrightarrow \left\{ f(x): \mathbb{N} \rightarrow \mathbb{R}^* \left| \begin{array}{l} (\exists k \in \mathbb{R}^+) \\ (\exists x_0 \in \mathbb{N}) \\ (\forall x \geq x_0) \\ [f(x) \leq kg(x)] \end{array} \right. \right\}$$

A function $f(x)$ is said to be in $O(g(x))$, denoted $f(x) = O(g(x))$, if $f(x)$ is bounded above by some constant multiple of $g(x)$ for all large x , i.e., if there exist some positive constant k and some nonnegative integer x_0 such that $f(x) \leq kg(x)$ for all $x \geq x_0$

For more explanation see the following figure

4.3) Asymptotic Notation (الترميز المقارب)

Big-O Notation:

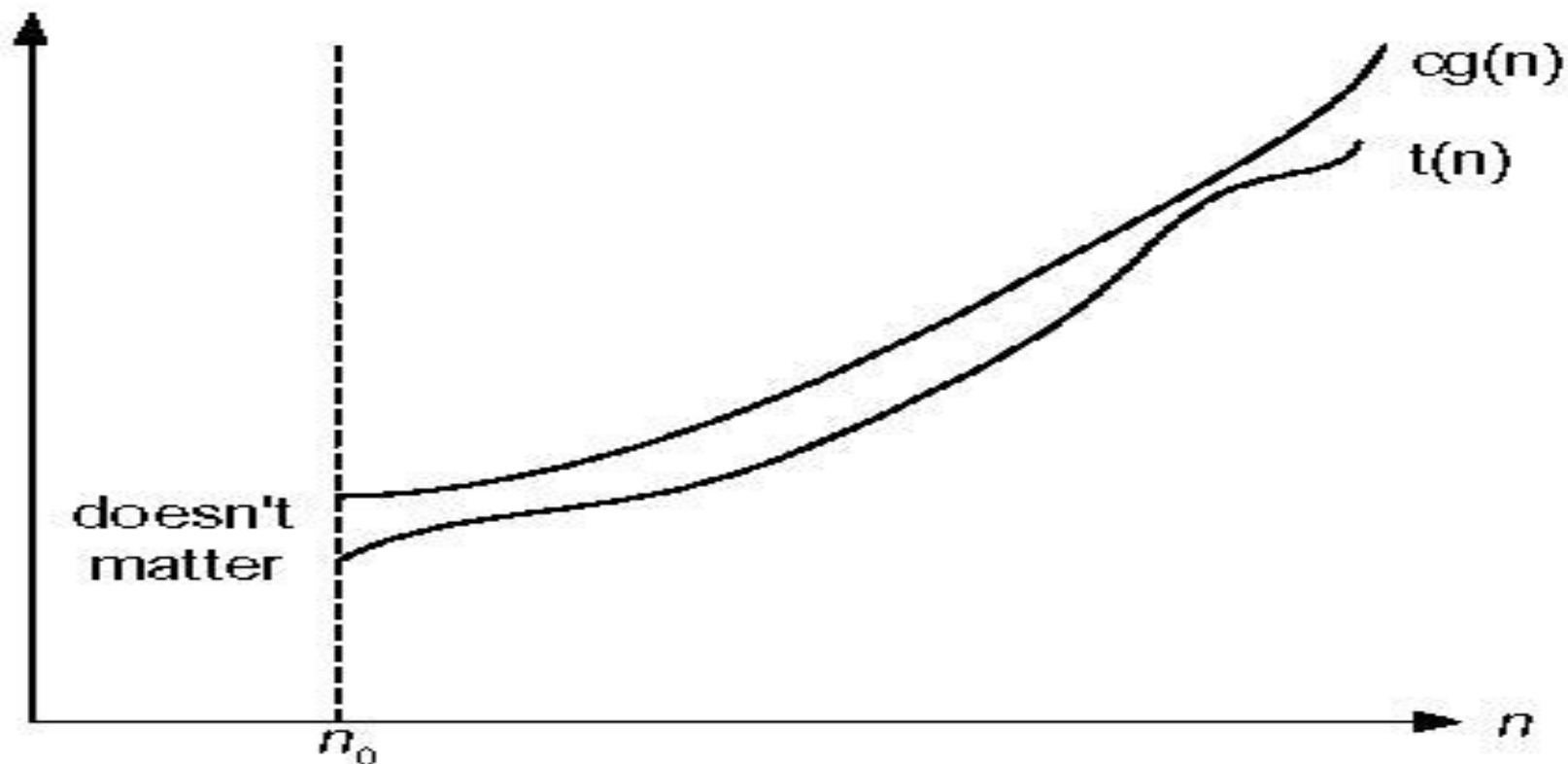


Figure 2.1 Big-oh notation: $t(n) \in O(g(n))$

4.3) Asymptotic Notation (الترميز المقارب)

Big-O Notation:

- Example (1)

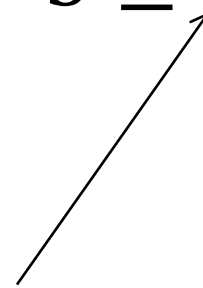
Prove that $n^4 + 2n^3 + 5 = O(n^4)$

Proof:

$$n^4 + 2n^3 + 5 \leq n^4 + 2n^4 + 5n^4$$

$$\therefore n^4 + 2n^3 + 5 \leq 8n^4$$

const



4.3) Asymptotic Notation (الترميز المقارب)

Big-O Notation:

- Example (2)

Give the Big-O estimate for $\sum_{j=1}^n j$

Solution:

$$\sum_{j=1}^n j \leq k \sum_{j=1}^n n = kn \times n = kn^2$$

$$\therefore \sum_{j=1}^n j \leq \underset{\substack{\uparrow \\ \text{const}}}{kn^2} \longrightarrow \sum_{j=1}^n j = O(n^2)$$

4.3) Asymptotic Notation (الترميز المقارب)

Rules of using of Big-O Notation:

1) Ignore the constant factors: $O(kg(x)) = O(g(x))$

Example: $O(20n^3) = O(n^3)$

2) Ignore the smaller terms:

IF $a < b$ **THEN** $O(a + b) = O(b)$

Example: $O(n^2 + n) = O(n^2)$

3) Take only the closed bound:

If f is bounded above by h and at the same time it bounded above by l and $h > l$ then actually $f = O(l)$

4.3) Asymptotic Notation (الترميز المقارب)

Rules of using of Big-O Notation:

Example:

$$n^4 + 2n^3 + 5 \leq 5n^5$$

$$\text{However, } n^4 + 2n^3 + 5 = O(n^4)$$

- 4) The functions n and $\log(n)$ are considered to be bigger than any constant. Therefore, k is a constant then $n + k = O(n)$, and similarly $\log(n) + k = O(\log(n))$

4.3) Asymptotic Notation (الترميز المقارب)

Big-O Notation:

■ Exercise:

Estimate the Big-O for the following functions:

- 10
- $n^3 + 2n^2 \log(n) + 8n^2$
- $(n^2 + 1)/(n + 1)$
- $\log(n^2 + 1)$
- $\sqrt{(n^2 + 1)}$

4.3) Asymptotic Notation (الترميز المقارب)

Big- Ω Notation:

- Denoted as follows

$$\Omega(g)$$

Such that g is a function

- Read as follows

In the order at least of g

- The Big-O notation is useful for estimating the upper limit of functions.
- It is also some times interesting to estimate a lower of the functions. Therefore, the **Ω -notation** is proposed for this purpose.

4.3) Asymptotic Notation (الترميز المقارب)

Big-Ω Notation:

It is formally defined as follows:

$$f(x) = \Omega(g(x)) \leftrightarrow \left\{ f(x): \mathbb{N} \rightarrow \mathbb{R}^* \left| \begin{array}{l} (\exists k \in \mathbb{R}^+) \\ (\exists x_0 \in \mathbb{N}) \\ (\forall x \geq x_0) \\ [f(x) \geq kg(x)] \end{array} \right. \right\}$$

A function $f(x)$ is said to be in $\Omega(g(x))$, denoted $f(x) = \Omega(g(x))$, if $f(x)$ is bounded below by some constant multiple of $g(x)$ for all large x , i.e., if there exist some positive constant k and some nonnegative integer x_0 such that $f(x) \geq kg(x)$ for all $x \geq x_0$

For more explanation see the following figure

4.3) Asymptotic Notation (الترميز المقارب)

Big- Ω Notation:

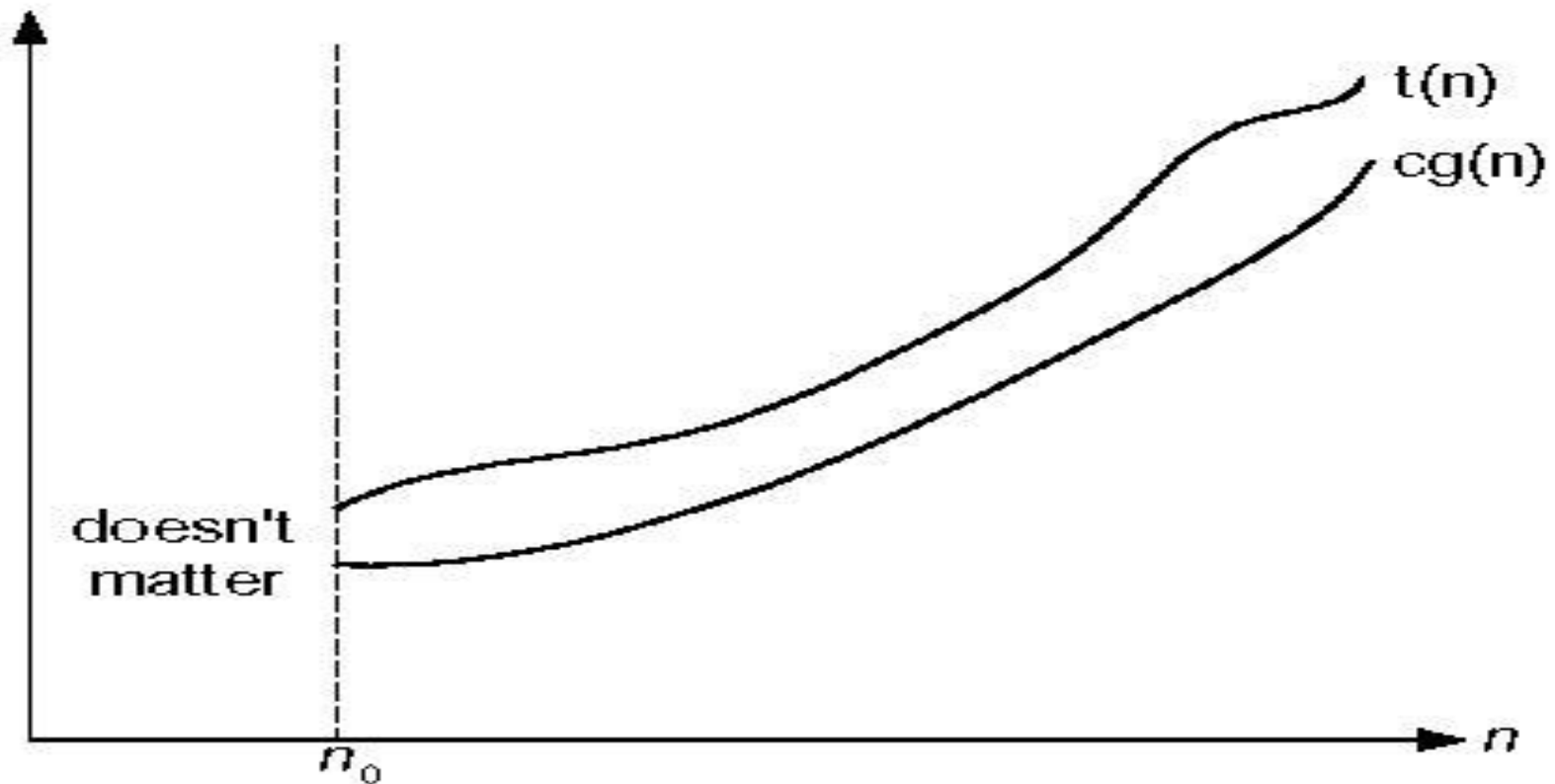


Fig. 2.2 Big-omega notation: $t(n) \in \Omega(g(n))$

4.3) Asymptotic Notation (الترميز المقارب)

Big-Ω Notation:

- Example (2)

Give the Ω estimate for $2n^2 + 5$

Solution:

$$2n^2 + 5 \geq \underset{\substack{\uparrow \\ \text{const}}}{1}n^2$$

$$\therefore 2n^2 + 5 = \Omega(n^2)$$

4.3) Asymptotic Notation (الترميز المقارب)

Θ -Notation:

- Denoted as follows

$$\Theta(g)$$

Such that g is a function

- Read as follows

In the exact order of g

4.3) Asymptotic Notation (الترميز المقارب)

Θ -Notation:

It is formally defined as follows:

A function $f(x)$ is said to be in $\Theta(g(x))$, denoted $f(x) = \Theta(g(x))$, if $f(x)$ is bounded both above and below by some constant multiple of $g(x)$ for all large x , i.e., if there exist some positive constant c_1 and c_2 and some nonnegative integer x_0 such that

$$c_1 g(x) \leq f(x) \leq c_2 g(x) \text{ for all } x \geq x_0$$

For more explanation see the following figure

4.3) Asymptotic Notation (الترميز المقارب)

Θ -Notation:

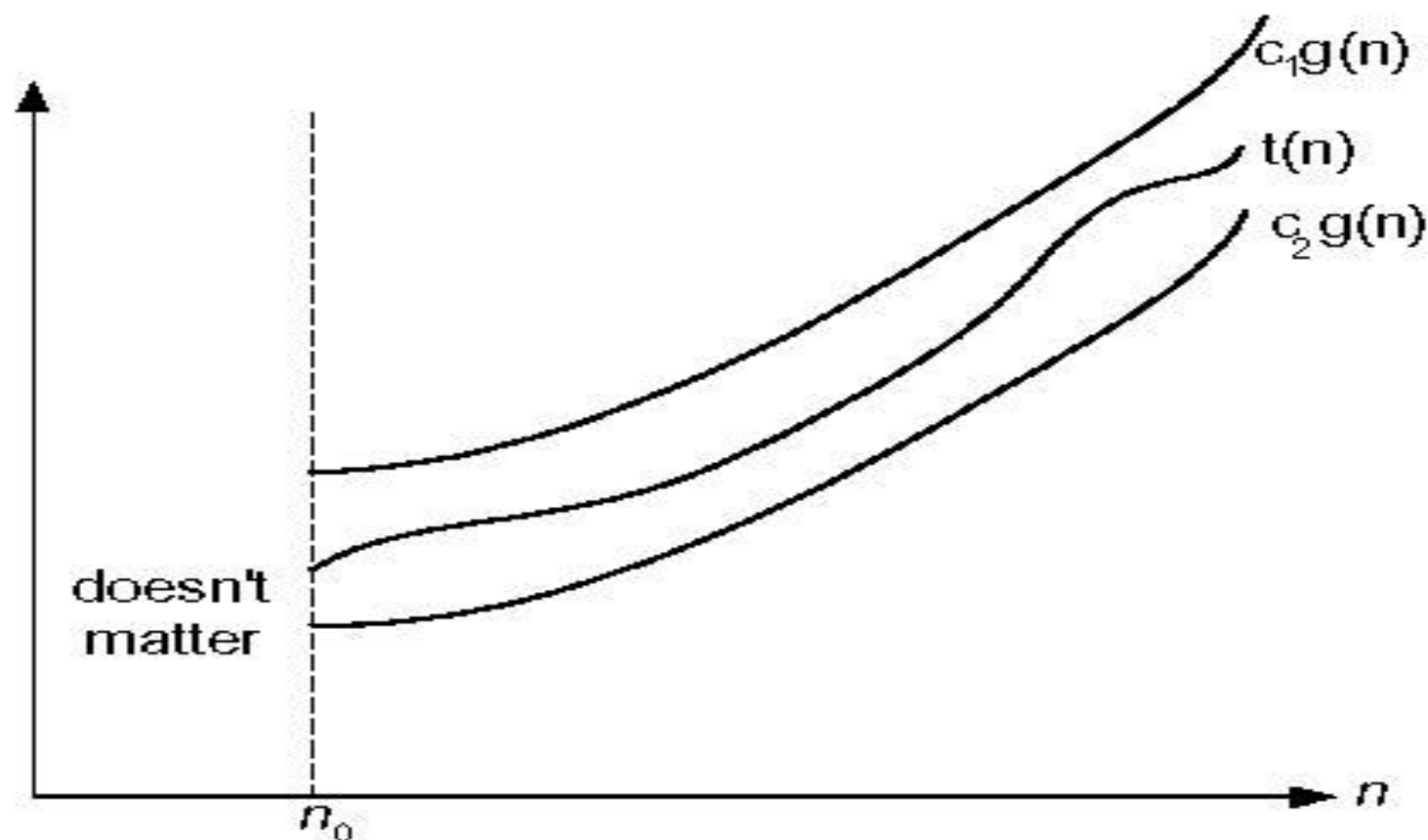


Figure 2.3 Big-theta notation: $t(n) \in \Theta(g(n))$

4.3) Asymptotic Notation (الترميز المقارب)

Θ-Notation:

■ Example (3)

Estimate the theta notation for $60n^2 + 5n + 1$

Solution:

$$60n^2 + 5n + 1 \leq n^2(60 + 5 + 1)$$

$$60n^2 + 5n + 1 \leq 66n^2$$

$$\therefore 60n^2 + 5n + 1 = O(n^2) \longrightarrow (1)$$

$$60n^2 + 5n + 1 \geq 60n^2$$

$$\therefore 60n^2 + 5n + 1 = \Omega(n^2) \longrightarrow (2)$$

$$\therefore 60n^2 + 5n + 1 = \Theta(n^2) \text{ based on (1) and (2)}$$

4.3) Asymptotic Notation (الترميز المقارب)

Θ -Notation:

- Exercise:

Prove that $2n + 3 \log(n) = \Theta(n)$

4.3) Asymptotic Notation (الترميز المقارب)

Other Notations:

- A function $f(n)$ is $O(g(n))$ if $\exists$ positive constants C and n_0 such that

$$f(n) < cg(n) \forall n \geq n_0$$

- A function $f(n)$ is $\omega(g(n))$ if $\exists$ positive constants C and n_0 such that

$$f(n) > cg(n) \forall n \geq n_0$$

- Intuitively,

$o(n)$ is like $<$ $\omega(n)$ is like $>$ $\Theta(n)$ is like $=$

$O(n)$ is like $\leq$ $\Omega(n)$ is like $\geq$

4.3) Asymptotic Notation (الترميز المقارب)

Measure Unit of Algorithm Theoretical Efficiency:

- It is natural to ask at this point *what* unit should be used to express the theoretical efficiency of an algorithm?

حتى هذه اللحظة هنالك سؤال يدور في أذهاننا وهو: ما هي الوحدة المستخدمة لقياس
لزمان الخوارزمية الناتج عن تحليلها نظرياً؟

- There can be no question of expressing algorithm efficiencies in terms of time measurements (such as seconds and minutes), since we do not have a standard computer to which all measurements might refer.

الحقيقة لا يمكننا أن نقول بأن مقاييس زمن تنفيذ الخوارزميات الناتج عن التحليل النظري
هي وحدات الزمن الحقيقي (مثل الدقائق والثواني)

- An answer to this question is given by the **principle of invariance**.

عليه فإن الإجابة على السؤال أعلاه تقدم بواسطة القاعدة التي تعرف
بالـ Principle of invariance

4.3) Asymptotic Notation (الترميز المقارب)

Measure Unit of Algorithm Theoretical Efficiency:

- According to this principle two different executions of the same algorithm will not differ in efficiency by more than some **multiplicative constant**.

على ضوء هذه القاعدة يمكننا القول بأن الاختلاف في Efficiency بين أي تنفيذين مختلفين لنفس الخوارزمية (أي أن الخوارزمية تم تنفيذها على جهازي حاسوب مختلفين في السرعة والمقدرات) هو ليس بأكثر من **معامل ضرب ثابت** (Multiplicative constant).

مثال:

✓ إذا كان لديك تنفيذين مختلفين لخوارزمية تعمل على حل Problem instance يكون فيه الـ Size مساوياً لـ n ، ووجدت أن هذين التنفيذين يأخذان زمن يقدر بـ $t_1(n)$ ثانية و $t_2(n)$ ثانية على التوالي

✓ عليه ستجد أن هنالك معامل ثابت موجب (c) على ضوءه نجد

$$t_1(n) \leq c t_2(n)$$

4.3) Asymptotic Notation (الترميز المقارب)

Measure Unit of Algorithm Theoretical Efficiency:

- Coming back to the question of the unit to be used to express the theoretical efficiency of an algorithm
 - ✓ There will be **no** such **unit**: we shall only **express** this **efficiency** to **within** a **multiplicative constant**
 - ✓ We say that an algorithm takes a time in the order of $t(n)$, for a given function t , if there exist a positive constant c and an implementation of the algorithm capable of solving every instance of the problem in a time bounded above by $ct(n)$ seconds, where n is the size
 - ✓ As we have seen previously, a more rigorous treatment of this important concept known as the **asymptotic notation**
- Actually, the **asymptotic notation** expresses the **execution time** of an algorithm in order of a **mathematical function** (i.e., **Big-O notation** or θ -notation)

4.3) Asymptotic Notation (الترميز المقارب)

Measure Unit of Algorithm Theoretical Efficiency:

- بالرجوع الى السؤال المتعلق بالوحدة المستخدمة لقياس زمن تنفيذ الخوارزميات المقدر نظرياً
 - ✓ ليس هنالك وحدة تستخدم لهذا الغرض، حيث يمكننا فقط التعبير عن هذا الزمن بواسطة معامل ضرب ثابت (Multiplicative Constant)
 - ✓ لذلك نقول أن زمن تنفيذ الخوارزمية يأخذ الرتبة $t(n)$ للدالة t ، عندما يوجد هنالك ثابت موجب c وأن زمن تنفيذ الخوارزمية يحد من أعلا بواسطة $ct(n)$ لكل قيم n
 - ✓ وبصورة أكثر دقة يتم عرض هذا المفهوم باستخدام الـ Asymptotic Notation
- عليه يقوم الـ Asymptotic Notation بالتعبير عن زمن تنفيذ الخوارزميات في شكل نظم لدوال رياضية وذلك باستخدام الـ Θ -Notation أو الـ Big-O notation

4.4) Asymptotic Notation as a measure of Algorithm Efficiency (استخدام الترميز المقارب كمقياس لزمن تنفيذ الخوارزميات)

- The asymptotic notation expresses the execution time of an algorithm in order of a mathematical function (i.e., Big-O notation and θ -notation).
 - ✓ This measure does not show the actual estimate of the execution time. But it shows that the time increases with the size, similarly as the function increases with its input.
- Therefore, this measure can estimate the **best-case time**, the **worst-case time**, or the **average-case time** of algorithms

- يشير المصطلح Best-Case Time للخوارزمية الى أفضل زمن (وهو أقل زمن) لتنفيذ الخوارزمية بين عدة تنفيذات لهذه الخوارزمية، حيث يستخدم كل واحد من هذه التنفيذات Instance مختلف عن الـ Instances المستخدمة في بقية التنفيذات
- كذلك يشير المصطلح Worst-Case Time للخوارزمية الى أسوأ زمن (وهو أكبر زمن) لتنفيذ الخوارزمية بين عدة تنفيذات لهذه الخوارزمية، حيث يستخدم كل واحد من هذه التنفيذات Instance مختلف عن الـ Instances المستخدمة في بقية التنفيذات
- المصطلح Average-Case Time للخوارزمية فهو المتوسط لعدة أزمان الناتجة عن تنفيذ الخوارزمية عدة مرات، حيث يتم استخدام Instance مختلف عن الـ Instances المستخدمة في تلك التنفيذات

4.4) Asymptotic Notation as a measure of Algorithm Efficiency (استخدام الترميز المقارب كمقياس لزمان تنفيذ الخوارزميات)

Example:

Suppose that the **worst-case time** for an algorithm is

$$t(n) = 60n^2 + 5n + 1$$

Such that n is the size of the algorithm execute the algorithm.

n	$t(n) = 60n^2 + 5n + 1$	$60n^2$
10	6051	6000
100	600501	600000
1000	60005001	60000000
10000	6000050001	6000000000

The table depicts that the increase the term $60n^2$ is approximately equal to the increase of $t(n)$ for large n . In this sense, $t(n)$ grows like $60n^2$

4.4) Asymptotic Notation as a measure of Algorithm Efficiency (استخدام الترميز المقارب كمقياس لزمن تنفيذ الخوارزميات)

Example:

The previous table depicts that the increase the term $60n^2$ is approximately equal to the increase of $t(n)$ for large n . In this sense, $t(n)$ grows like $60n^2$

Therefore, we can say that **worst-case time** for the algorithm (which notated as $t(n)$) is **in the exact order of n^2** , and we write it as follows:

$$t(n) = \Theta(n^2)$$

وهذا يعني أن الزيادة زمن الخوارزمية t الناتجة عن الزيادة في n تكون شبيهة بالزيادة في منحنى الدالة n^2

4.4) Asymptotic Notation as a measure of Algorithm Efficiency (استخدام الترميز المقارب كمقياس لزمن تنفيذ الخوارزميات)

Orders of Growth for some functions:

الجدول التالي يبين قيم الزيادة لبعض الدوال الناتجة عن الزيادة في Input

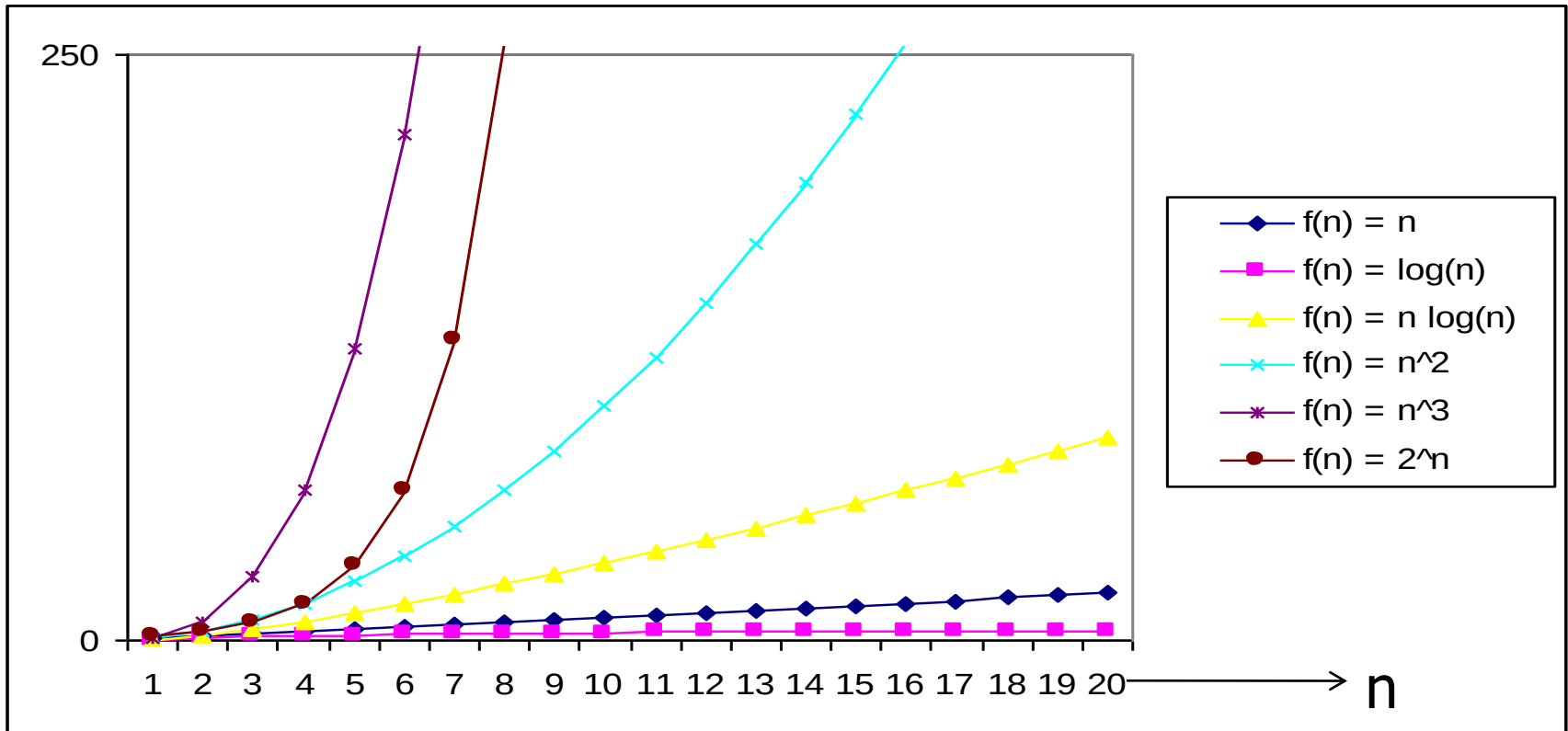
n	$\log_2 n$	n	$n \log_2 n$	n^2	n^3	2^n	$n!$
10	3.3	10^1	$3.3 \cdot 10^1$	10^2	10^3	10^3	$3.6 \cdot 10^6$
10^2	6.6	10^2	$6.6 \cdot 10^2$	10^4	10^6	$1.3 \cdot 10^{30}$	$9.3 \cdot 10^{157}$
10^3	10	10^3	$1.0 \cdot 10^4$	10^6	10^9		
10^4	13	10^4	$1.3 \cdot 10^5$	10^8	10^{12}		
10^5	17	10^5	$1.7 \cdot 10^6$	10^{10}	10^{15}		
10^6	20	10^6	$2.0 \cdot 10^7$	10^{12}	10^{18}		

Table 2.1 Values (some approximate) of several functions important for analysis of algorithms

4.4) Asymptotic Notation as a measure of Algorithm Efficiency (استخدام الترميز المقارب كمقياس لزمن تنفيذ الخوارزميات)

Orders of Growth for some functions:

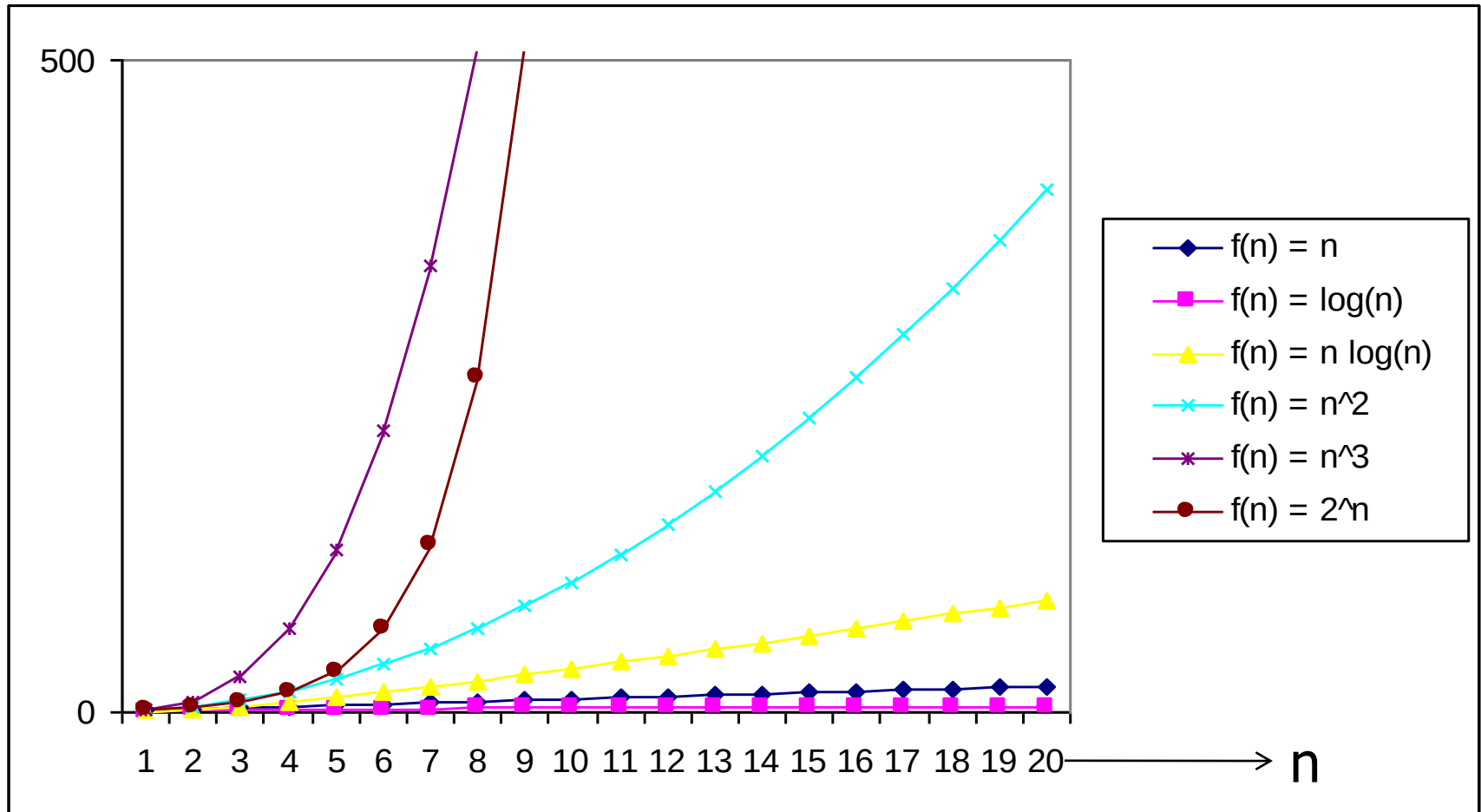
ويبين الرسم التالي منحنيات الزيادة لبعض الدوال الناتجة عن الزيادة في Input



A Graph that depicts the grows of some functions based on the increase of n

4.4) Asymptotic Notation as a measure of Algorithm Efficiency (استخدام الترميز المقارب كمقياس لزمن تنفيذ الخوارزميات)

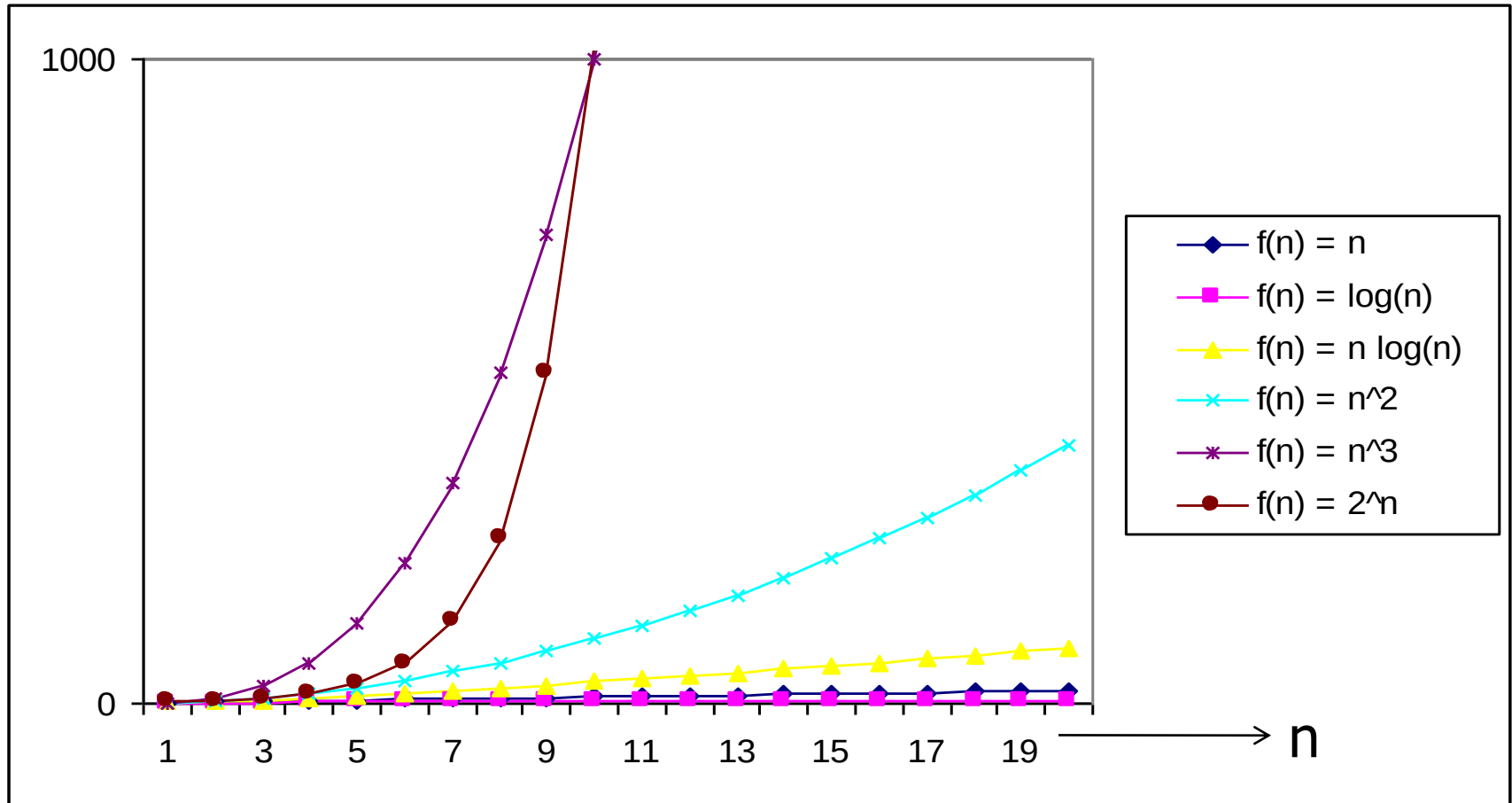
Orders of Growth for some functions:



A Graph that depicts the grows of some functions based on the increase of n

4.4) Asymptotic Notation as a measure of Algorithm Efficiency (استخدام الترميز المقارب كمقياس لزمان تنفيذ الخوارزميات)

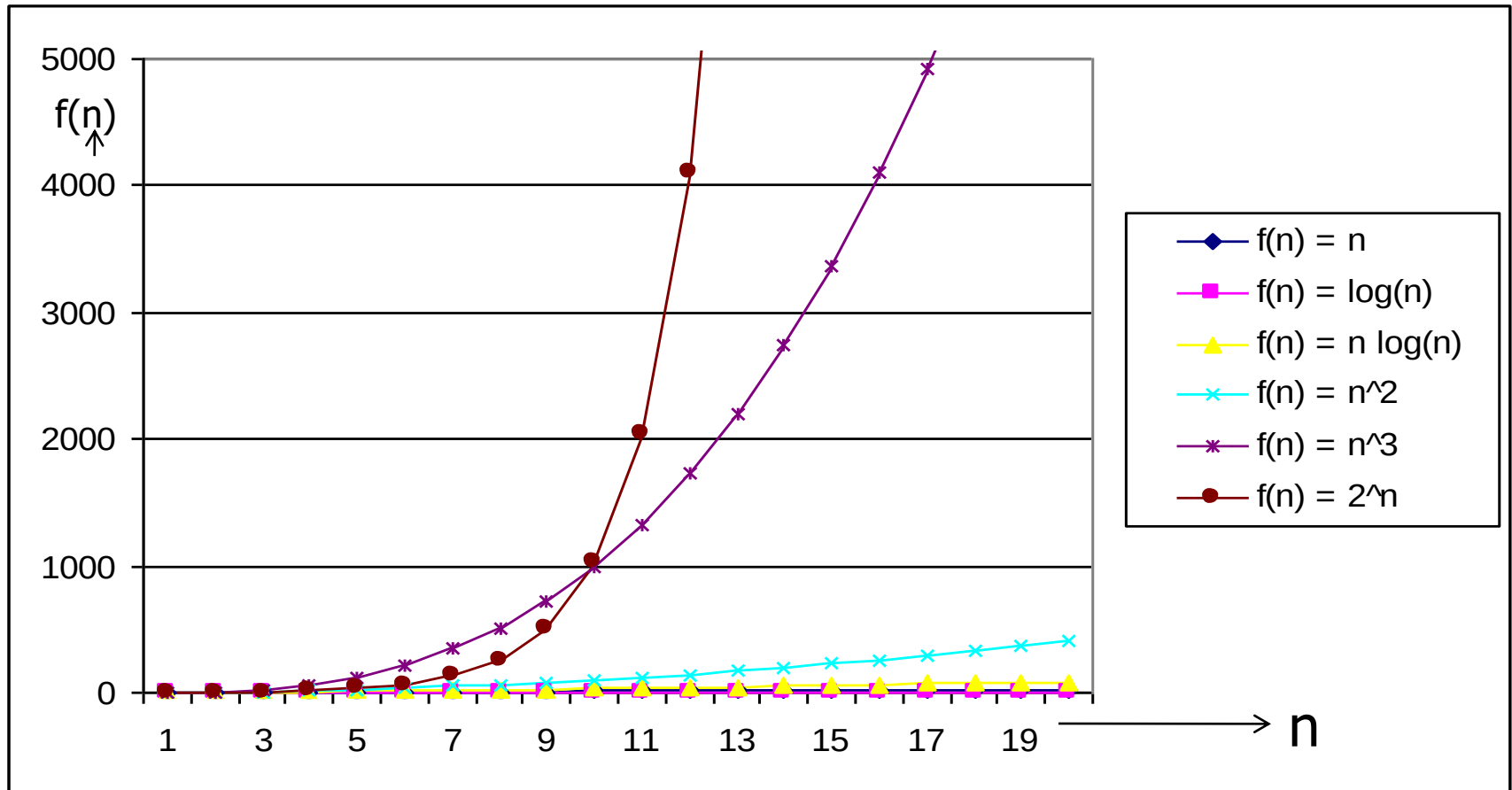
Orders of Growth for some functions:



A Graph that depicts the grows of some functions based on the increase of n

4.4) Asymptotic Notation as a measure of Algorithm Efficiency (استخدام الترميز المقارب كمقياس لزمان تنفيذ الخوارزميات)

Orders of Growth for some functions:



A Graph that depicts the grows of some functions based on the increase of n